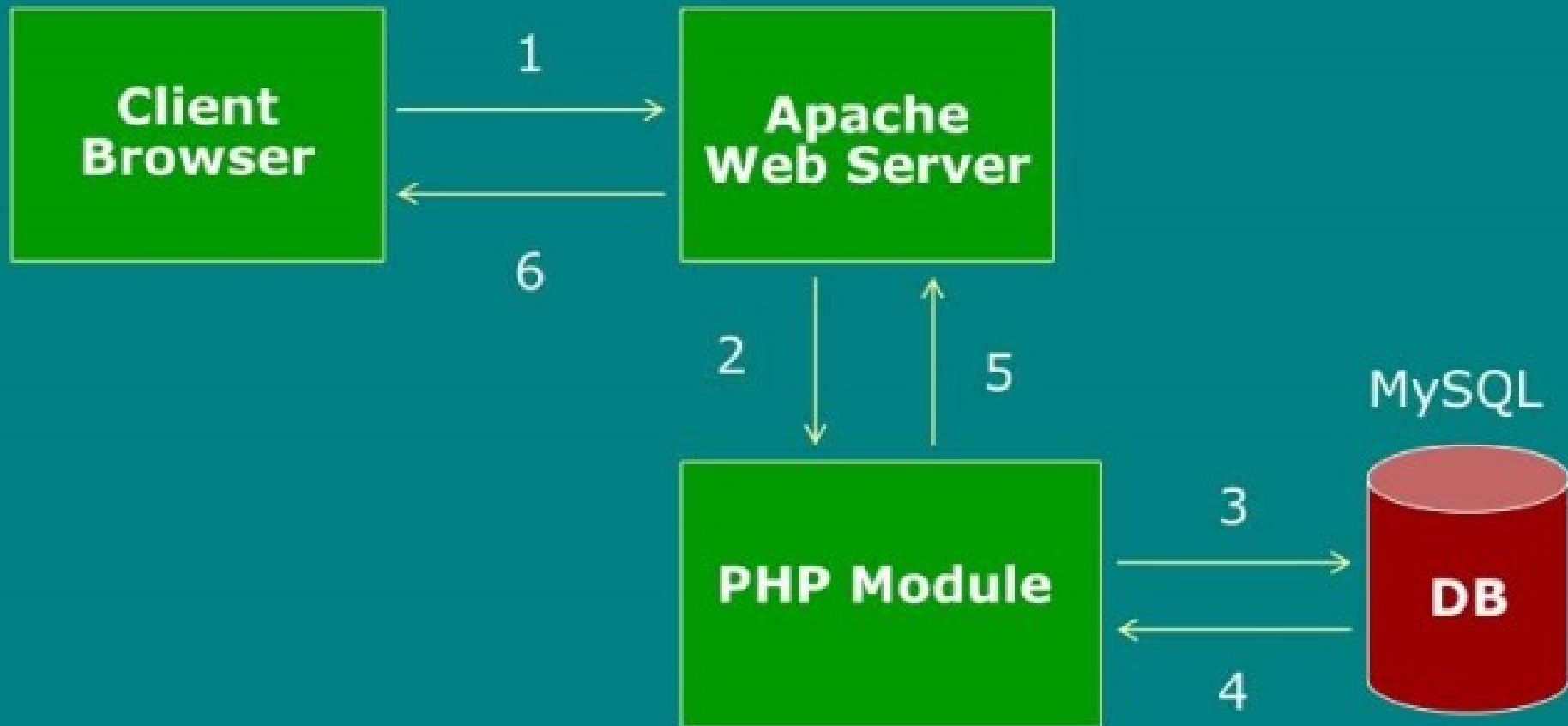

Chapter three

Connecting to Databases with PHP

Accessing a MySQL Database with PHP Script



Connecting to Database

- PHP 5 and later can work with a MySQL database using:
 - » **MySQLi extension** (the "i" stands for improved)
 - » **PDO (PHP Data Objects)**
- PDO will work on **12 different database systems**, whereas MySQLi will **only work with MySQL databases**.
- So, if you have to switch your project to use another database, **PDO** makes the process easy.
- You only have to change the connection string and a few queries.
- With MySQLi, you will need to rewrite the entire code queries included.
- Both are object-oriented, but MySQLi also offers a procedural API.

Connecting...

- The first step for interacting with MySQL connecting to the server requires the appropriately named **mysqli_connect()** function:
 - » `$dbc = mysqli_connect (hostname, username, password, db_name);`
- The first three arguments sent to the function (***hostname, username, and password***) are based upon the users and privileges established within MySQL.
- The fourth argument is the name of the database to use. This is the equivalent of saying **USE databasename** within the mysql client.
- If the connection was made, the **\$dbc** variable, short for database connection (but you can use any name you want, of course).

Connecting...

- Most of the PHP functions for working with MySQL will take this variable as its first argument.
- If a connection problem occurred, you can call **mysqli_connect_error()**, which returns the connection error message.
- It takes ***no arguments***, so would be called using just
 - » **mysqli_connect_error();**
- Set the MySQL host, username, password, and database name as constants:
 - » **DEFINE ('DB_USER', 'username');**
 - » **DEFINE ('DB_PASSWORD', 'password');**
 - » **DEFINE ('DB_HOST', 'localhost');**
 - » **DEFINE ('DB_NAME', 'sitename');**

Connecting...

- Example: (Object Oriented)

- <?php

```
$servername = "localhost";  
$username = "username";  
$password = "password";
```

```
// Create connection
```

```
$dbc = new mysqli($servername, $username,  
$password);
```

```
// Check connection
```

```
if ($dbc->connect_error) {  
    die("Connection failed: " . $dbc->connect_error);  
}
```

```
echo "Connected successfully";
```

```
?>
```

Connecting...

- Example: (Procedural)

- ```
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection
$dbconn = mysqli_connect($servername, $username,
$password);

// Check connection
if (!$dbconn) {
 die("Connection failed: " .
mysqli_connect_error());
}
echo "Connected successfully";
?>
```

# Connecting...

- Example: (PDO)

- ```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
try {
    $dbc = new
PDO("mysql:host=$servername;dbname=myDB", $username,
$password);
    echo "Connected successfully";
}
catch(PDOException $e)
{
    echo $e->getMessage();
}
?>
```


Close the Connection

- `$dbc->close();` - *for object oriented*
- `mysqli_close($dbc);` - *for procedural*
- `$dbc = null;` - *for PDO*

Create a MySQL Database

- The CREATE DATABASE statement is used to create a database in MySQL.
- ```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
// Create connection
$conn = new mysqli($servername, $username,
$password);
// Check connection
if ($conn->connect_error) {
 die("Connection failed: " . $conn-
>connect_error);
}
```

# Create...

- ```
// Create database
$sql = "CREATE DATABASE myDB";
if ($conn->query($sql) === TRUE) {
    echo "Database created successfully";
} else {
    echo "Error creating database: " .
    $conn->error;
}

$conn->close();
?>
```

Create Table

- The CREATE TABLE statement is used to create a table in MySQL.
- We will create a table named "MyGuests", with five columns: "id", "firstname", "lastname", "email" and "reg_date":
- ```
CREATE TABLE MyGuests (
 id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY
 KEY,
 firstname VARCHAR(30) NOT NULL,
 lastname VARCHAR(30) NOT NULL,
 email VARCHAR(50),
 reg_date TIMESTAMP
)
```

# Create...

- The data type specifies what type of data the column can hold.
- After the data type, you can specify other optional attributes for each column:
  - » NOT NULL - Each row must contain a value for that column, null values are not allowed
  - » DEFAULT value - Set a default value that is added when no other value is passed
  - » UNSIGNED - Used for number types, limits the stored data to positive numbers and zero
  - » AUTO INCREMENT - MySQL automatically increases the value of the field by 1 each time a new record is added
  - » PRIMARY KEY - Used to uniquely identify the rows in a table. The column with PRIMARY KEY setting is often an ID number, and is often used with AUTO\_INCREMENT
- Each table should have a primary key column (in this case: the "id" column). Its value must be unique for each record in the table.

# Create...

- **Example: (Object oriented)**

- ```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username,
$password, $dbname);
// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn-
>connect_error);
}
```

Create...

```
// sql to create table
$sql = "CREATE TABLE MyGuests (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
firstname VARCHAR(30) NOT NULL,
lastname VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP
)";

if ($conn->query($sql) === TRUE) {
    echo "Table MyGuests created successfully";
} else {
    echo "Error creating table: " . $conn->error;
}

$conn->close();
?>
```

Sending data into Database

- After a database and a table have been created, we can start adding data in them.
- Here are some syntax rules to follow:
 - » The SQL query must be quoted in PHP
 - » String values inside the SQL query must be quoted
 - » Numeric values must not be quoted
 - » The word NULL must not be quoted
- The INSERT INTO statement is used to add new records to a MySQL table:
 - » `INSERT INTO table_name (column1, column2, column3,...)
VALUES (value1, value2, value3,...)`

Sending...

- **Example:**

- ```
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username,
$password, $dbname);
// Check connection
if ($conn->connect_error) {
 die("Connection failed: " . $conn-
>connect_error);
}
```

# Sending...

- ```
$sql = "INSERT INTO MyGuests (firstname, lastname,  
email)  
VALUES ('John', 'Doe', 'john@example.com')";  
  
if ($conn->query($sql) === TRUE) {  
    echo "New record created successfully";  
} else {  
    echo "Error: " . $sql . "<br>" . $conn->error;  
}  
  
$conn->close();  
?>
```

Sending...

- Multiple SQL statements must be executed with the `mysqli_multi_query()` function.
- ```
$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com');";
$sql .= "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Mary', 'Moe', 'mary@example.com');";
$sql .= "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('Julie', 'Dooley', 'julie@example.com');";

if ($conn->multi_query($sql) === TRUE) {
 echo "New records created successfully";
} else {
 echo "Error: " . $sql . "
" . $conn->error;
}
```

# Sending...

- If we perform an INSERT or UPDATE on a table with an AUTO\_INCREMENT field, we can get the ID of the last inserted/updated record immediately.
- ```
$sql = "INSERT INTO MyGuests (firstname, lastname,  
email)  
VALUES ('John', 'Doe', 'john@example.com')";  
  
if ($conn->query($sql) === TRUE) {  
    $last_id = $conn->insert_id;  
    echo "New record created successfully. Last  
inserted ID is: " . $last_id;  
} else {  
    echo "Error: " . $sql . "<br>" . $conn->error;  
}
```

Retrieve Data from a Database

- The SELECT statement is used to select data from one or more tables:
- SELECT column_name(s) FROM table_name
- SELECT * FROM table_name
- ```
$sql = "SELECT id, firstname, lastname FROM
MyGuests";
$result = $conn->query($sql);
if ($result->num_rows > 0) {
 // output data of each row
 while($row = $result->fetch_assoc()) {
 echo "id: " . $row["id"]. " - Name: " .
$row["firstname"]. " " . $row["lastname"]. "
";
 }
} else {
 echo "0 results";
}
```

# Retrieve...

- MySQL provides a LIMIT clause that is used to specify the number of records to return.
- The LIMIT clause makes it easy to code multi page results or pagination with SQL, and is very useful on large tables.
- Returning a large number of records can impact on performance.
  - » `$sql = "SELECT * FROM Orders LIMIT 30";` // it will return the first 30 records.
  - » `$sql = "SELECT * FROM Orders LIMIT 10 OFFSET 15";` OR
  - » `$sql = "SELECT * FROM Orders LIMIT 15, 10";` // return only 10 records, start on record 16 (OFFSET 15)“ OR select records 16 - 25

# Modify Existing Data

- The UPDATE statement is used to update existing records in a table:
- UPDATE table\_name  
SET column1=value, column2=value2,...  
WHERE some\_column=some\_value
- ```
$sql = "UPDATE MyGuests SET lastname='Doe' WHERE  
id=2";  
  
if ($conn->query($sql) === TRUE) {  
    echo "Record updated successfully";  
} else {  
    echo "Error updating record: " . $conn->error;  
}
```

Remove Existing Data

- The DELETE statement is used to delete records from a table:
- DELETE FROM table_name
WHERE some_column = some_value
- ```
// sql to delete a record
$sql = "DELETE FROM MyGuests WHERE id=3";

if ($conn->query($sql) === TRUE) {
 echo "Record deleted successfully";
} else {
 echo "Error deleting record: " . $conn->error;
}
```



# Form Handling and Data Validation with PHP

---

## 1. Basic HTML Form Structure

- First, we need an HTML form to collect user input.
- The form uses the POST method to send data securely to the PHP script for processing.

# Complete PHP Form with Validation and Sanitization

---

The form validation includes:

- ✓ Required field validation
- ✓ Email validation
- ✓ Name format validation
- ✓ Gender selection
- ✓ Message field validation
- ✓ Retaining user input after submission
- ✓ Success message upon submission

# How It Works

---

## 1. Validation Rules

- **Name:** Required, only letters and spaces allowed.
- **Email:** Required, must be a valid format.
- **Gender:** Required.
- **Message:** Optional, but limited to 250 characters.

## 2. Sanitization (`test_input()`)

- `trim()` → Removes unnecessary spaces.
- `stripslashes()` → Removes backslashes.
- `htmlspecialchars()` → Prevents HTML injection.

# Cont...

## 3. Error Messages

- Displayed next to invalid fields.
- If a field is empty, a red error message appears next to it.

## 4. Retaining Input

- Prevents user data from being lost after form submission.
- The value="<?php echo \$name; ?>" ensures that after submitting, the entered data remains.

## 5. Check for Required Fields:

if (empty(\$\_POST["name"])) → If the field is empty, it sets an error message.

## 6. Success Message

If all fields are valid, the following message is displayed:

# PHP Form Validation

- Before Submission:

**PHP Complete Form Validation**

Name:  \*

Email:  \*

Gender: ☐ Male ☐ Female \*

Message:

- If Name & Email Are Empty:

**PHP Complete Form Validation**

Name:  \* Name is required

Email:  \* Email is required

Gender: ☐ Male ☐ Female \* Gender is required

Message:

# PHP Form Validation

---

- If Name & Email Are not valid:

## PHP Complete Form Validation

Name:  \* Only letters and spaces allowed

Email:  \* Invalid email format

Gender: ☐ Male ☐ Female \* Gender is required

Message:

# PHP Form Validation

- After Successful Submission:

## PHP Complete Form Validation

Name:  \*

Email:  \*

Gender: ☐ Male ☒ Female \*

Message: 

Hi, this is the test message!

## Form Submitted Successfully

Name: Hiywot

Email: hiywotyesuf@gmail.com

Gender: Female

Message: Hi, this is the test message!



# Web Application Security

---

- It refers to the measures and practices implemented to protect web applications from various security threats and vulnerabilities.
- Since web applications are often exposed to the internet, they are vulnerable to attacks like:
  - SQL injection,
  - Cross-Site Scripting (XSS),
  - Cross-Site Request Forgery (CSRF), and more.
- Securing a web application is essential to protect user data, prevent unauthorized access, and ensure the application functions as intended.

# Security methods for Web Application

---

## 1. Authentication and Authorization

- Authentication verifies the identity of a user
- authorization determines what an authenticated user is allowed to do.

Best mechanism for strong authentication

- **Strong Passwords:** Enforce strong password policies, e.g., at least 8 characters, a mix of letters, numbers, and symbols.
- **Two-Factor Authentication (2FA):** Implement 2FA to add an additional layer of security.
- **Password Hashing:** Use secure hashing algorithms (e.g., bcrypt, Argon2) to store passwords securely in the database, instead of storing them as plain text.

*Example using bcrypt*

```
$hashedPassword = password_hash($password, PASSWORD_BCRYPT);
```

- Session Management: Use secure session management techniques, such as:
  - Regenerating session IDs upon login to prevent session fixation attacks.
  - Setting secure flags for session cookies (e.g., HttpOnly, Secure).

# Web Application Security

---

## 2. Input Validation and Sanitization

- User inputs should always be validated and sanitized to prevent malicious data from being processed by the server.

Best Methods:

- Sanitize Inputs: Remove or escape unwanted characters from user input.  
For example, use `htmlspecialchars()` to prevent XSS.  
`$safe_input = htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');`
- Validate Inputs: Ensure that the input meets the expected format.

For example

- Use `filter_var()` to validate email addresses.
- Use regular expressions to validate phone numbers or passwords.

```
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
 echo "Invalid email format";
}
```

# Web Application Security

## 3. SQL Injection Prevention

- SQL Injection occurs when a malicious user injects SQL code into a query, allowing them to manipulate the database.

Best method:

- Prepared Statements: Always use prepared statements with bound parameters. This is supported by PDO and MySQLi.

// Example using PDO

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE email = :email");
```

```
$stmt->bindParam(':email', $email, PDO::PARAM_STR);
```

```
$stmt->execute();
```

- Avoid Direct SQL Concatenation: Never concatenate user input directly into SQL queries. Instead, use prepared statements as shown above.

# Web Application Security

---

## 4. Cross-Site Scripting (XSS)

XSS attacks involve injecting malicious scripts into web pages that are then executed in the browsers of other users.

- **Output Encoding:** Always escape user input before displaying it in the browser using functions like htmlspecialchars().  
`echo htmlspecialchars($user_input, ENT_QUOTES, 'UTF-8');`
- **Use Content Security Policy (CSP):** Define a CSP header to restrict the types of content that can be loaded and executed by the browser.  
`Content-Security-Policy: script-src 'self' https://trusted.com;`
- **Sanitize HTML:** If users are allowed to enter HTML content, sanitize it to remove potentially dangerous tags (e.g., <script>).

# Web Application Security

## 5. Cross-Site Request Forgery (CSRF)

- **CSRF attacks** involve tricking a user into performing actions they didn't intend to, such as submitting a form or clicking a link that performs a harmful action on a site where they are authenticated.

Best Method:

- **CSRF Tokens:** Include a unique, random token in forms to validate that the request originated from the user's session.

Example in PHP:

```
// Generate a CSRF token
$_SESSION['csrf_token'] = bin2hex(random_bytes(32));
// Include CSRF token in forms
echo '<input type="hidden" name="csrf_token" value="' . $_SESSION['csrf_token'] . '">';
// Validate CSRF token
if ($_POST['csrf_token'] !== $_SESSION['csrf_token']) {
 die('CSRF token mismatch');}
```

- **SameSite Cookies:** Set the SameSite cookie attribute to prevent browsers from sending cookies with cross-site requests.  
setcookie('SESSIONID', \$session\_id, ['samesite' => 'Strict']);

# Web Application Security

## 6. File Upload Security

- Allowing file uploads opens your application to several security risks, such as malicious files (e.g., scripts) being uploaded to the server.

Best method:

- **Limit File Types:** Only allow specific file types (e.g., images, PDFs) by checking the file MIME type.  

```
if (!in_array($file['type'], ['image/jpeg', 'image/png'])) {
 die('Invalid file type');
}
```
- **Limit File Size:** Enforce size limits for uploads to prevent large, potentially malicious files from being uploaded.
- **Rename Files:** Do not allow users to upload files with their original names. Rename files upon upload to avoid executing dangerous scripts.
- **Store Files Outside Web Root:** Save uploaded files outside the web-accessible directory to prevent direct access.

# Web Application Security

---

## 7. Secure HTTP Headers

- HTTP headers can be used to enforce security policies in your web application.

Best method:

- Strict-Transport-Security (HSTS): Enforce HTTPS connections for your website.

**Strict-Transport-Security: max-age=31536000; includeSubDomains;**

- X-Content-Type-Options: Prevent the browser from interpreting files as a different MIME type.

**X-Content-Type-Options: nosniff**

- X-Frame-Options: Prevent your site from being embedded in iframes (mitigates clickjacking attacks).

**X-Frame-Options: SAMEORIGIN**

- X-XSS-Protection: Enable built-in XSS protection in browsers.

**X-XSS-Protection: 1; mode=block**



# Web Application Security

---

## 8. Security Logging and Monitoring

- It's crucial to monitor your web application for suspicious activity and security incidents.

Best method:

- **Logging:** Log critical events, such as login attempts, errors, and failed authorization attempts. Make sure logs don't contain sensitive information (e.g., passwords).
- **Intrusion Detection:** Implement intrusion detection mechanisms to monitor for signs of attacks (e.g., abnormal traffic patterns, SQL injection attempts).
- **Error Handling:** Do not display detailed error messages in production. Instead, log errors on the server and show generic messages to users.

# Web Application Security

---

## 9. Security Testing

- Security testing should be a regular part of your development process to identify vulnerabilities before deployment.

Best mechanism:

- **Penetration Testing:** Regularly conduct penetration tests to identify vulnerabilities.
- **Static and Dynamic Code Analysis:** Use tools that analyze your code (e.g., static analysis for detecting vulnerabilities) and your running application (dynamic analysis).

## 10. Security Best Practices for Deployment

- **Keep Software Updated:** Regularly update all software components (web server, PHP, libraries, etc.) to ensure they are free from known vulnerabilities.
- **Use Firewalls:** Deploy firewalls to protect your application from external threats.
- **Backup and Recovery:** Implement a regular backup strategy and a recovery plan in case of a breach or data loss.

# Data base security using server side scripting

---

- Database security with respect to PHP comes down to **three broad issues**:
  1. Protecting the MySQL access information
  2. Not revealing too much about the database
  3. Being cautious when running queries, particularly those involving usersubmitted data
- You can accomplish the **first objective** by securing the MySQL connection script outside of the Web directory so that it is never viewable through a Web browser.
- The **second objective** is attained by not letting the user see PHP's error messages or your queries (in these scripts, that information is printed out for your debugging purposes; you'd never want to do that on a live site).

# Database Security...

- For the third objective, there are numerous steps you can and should take, all based upon the premise of never trusting user-supplied data.
  - » First, validate that some value has been submitted, or that it is of the proper type (number, string, etc.).
  - » Second, use regular expressions to make sure that submitted data matches what you would expect it to be.
  - » Third, you can typecast some values to guarantee that they're numbers.
  - » A fourth recommendation is to run user-submitted data through the **mysqli\_real\_escape\_string( )** function.
- This function makes data safe to use in a query by escaping what could be problematic characters.
  - » **\$safe = mysqli\_real\_escape\_string (\$dbc, *data*);**